PL/SQL

- PL/SQL (Procedural Language/SQL)
  - Is the Oracle procedural extension of SQL
  - Is a portable, high-performance transaction-processing language
  - Is a proprietary language

PL/SQL

- Advantages of PL/SQL:
  - Tight Integration with SQL
  - High Performance
  - High Productivity
  - Portability
  - Scalability
  - Manageability
  - Support for Object-Oriented Programming

PL/SQL
Tight Integration with SQL

- PL/SQL supports
  - SQL data manipulation statements DML
  - Cursor control statements (OPEN, LOOP, CLOSE)
  - Transaction control statements TCL

PL/SQL
Tight Integration with SQL

- DML (Data Manipulation Language)
  - SELECT
  - INSERT
  - UPDATE
  - DELETE
- TCL (Transaction Control Language)
  - COMMIT
  - ROLLBACK
  - SAVEPOINT

PL/SQL
Tight Integration with SQL

- PL/SQL supports
  - All Oracle SQL Functions
  - All SQL Operators
  - Pseudocolumns
    - A **pseudocolumn** behaves like a table column but is not actually stored in the table
    - E.g., COLUMN_VALUE, **ROWID**, **ROWNUM**, XMLDATA

PL/SQL
Tight Integration with SQL

- PL/SQL supports all SQL data types
  - No need for conversions between PL/SQL and SQL data types
  - Attributes for data type correlation
    - %ROWTYPE attribute to declare a record that represents either a full or partial row of a database table or view
    - %TYPE attribute to declare a data item of the same data type as a previously declared variable or column

PL/SQL
Tight Integration with SQL

- PL/SQL supports
  - Running SQL query
  - Processing the rows of the result set one at a time
- PL/SQL functions can be declared and defined in the WITH clauses of SQL SELECT statements (CTE – Common Table Expressions)

Examples

```
with no_emps as (
    select department_id, count(*) c
    from employees
    group by department_id
)
select d.department_name, q.c  emps
from departments d
    inner join no_emps q
        on q.department_id = d.department_id
order by 1;

-- Equivalent

select
    d.department_name,
    count(e.employee_id) emps
from departments d
    inner join employees e
        on d.department_id=e.department_id
group by d.department_name
order by 1;
```

PL/SQL
Tight Integration with SQL

- PL/SQL supports
  - Static SQL
    - SQL whose full text is known at compile time
  - Dynamic SQL
    - SQL whose full text is not known until run time
    - Make applications more
      - Flexible
      - Versatile

PL/SQL
Tight Integration with SQL

- Dynamic SQL Drawbacks
  - Performance loss
    - The execution plan for dynamic queries cannot be cached
  - Hard to debug
    - The error management becomes more unreliable.
    - Maintenance is difficult because the schema is hard coded in the dynamic code.
  - Security can be compromised with SQL injection

PL/SQL
High Performance

- PL/SQL
  - Statements can be organized into blocks (BEGIN … END)
  - Blocks significantly reduce traffic between the application and the database.
- High performance is achieved through:
  - Bind Variables
  - Subprograms
  - Optimizer

PL/SQL
High Performance

- Bind Variables
  - Are placeholders for actual values in SQL statements
  - Allow the database server to prepare the statement once and execute it multiple times without reparsing or reanalyzing it.
  - The PL/SQL compiler turns the variables in the WHERE and VALUES clauses into bind variables when embedding a SQL data manipulation statements directly in a block
  - Oracle Database can reuse these SQL statements each time the same code runs
  - PL/SQL does not create bind variables automatically when you use dynamic SQL
    - They can be used with dynamic SQL by specifying them explicitly

PL/SQL
High Performance

- PL/SQL supports subprograms:
  - Functions
  - Procedures
- Subprograms
  - Are stored in executable form
  - Can be invoked repeatedly
  - Are cached lowering memory requirements
  - Are shared among users lowering invocation overhead
  - A single invocation over the network can start a large job
    - Reducing network traffic
    - Improving response times
- Optimizer
  - The PL/SQL compiler has an optimizer that can rearrange code for better performance.

PL/SQL
High Productivity

- PL/SQL
  - Is the same in all environments
  - Can be used to write compact code for manipulating data
  - Is a scripting language
  - Can query, transform, and update data in a database
  - Supported by many other Oracle tools (Forms Builder, Apex, etc.)

PL/SQL
Portability

- PL/SQL is a portable and standard language for Oracle development
- PL/SQL applications can be run on any operating system and platform where Oracle Database runs
- PL/SQL applications can be run on different environments (e.g., server, client application, etc.)

PL/SQL
Scalability

- PL/SQL stored subprograms increase scalability by centralizing application processing on the database server
- The shared memory facilities of the shared server let Oracle Database support thousands of concurrent users on a single node
- Oracle Connection Manager can be used to multiplex network connections

PL/SQL Manageability

- PL/SQL stored subprograms increase manageability:
  - Only one copy of a subprogram can be maintained on the database server
  - Rather than one copy on each client system
- Any number of applications can use the subprograms
- Subprograms can be changed without affecting the applications that invoke them

PL/SQL Support for Object-Oriented Programming

- PL/SQL supports OOP
  - Allows defining object types that can be used in object-oriented designs
  - Abstract Data Types (ADT):
    - Consists of a data structure and subprograms that manipulate the data
    - The variables that form the data structure are called attributes.
    - The subprograms that manipulate the attributes are called methods.

PL/SQL Main Features

- PL/SQL combines the data-manipulating power of SQL with the processing power of procedural languages
- SQL statements can be issued from a PL/SQL program
- Complex problems can be broken into easily understandable subprograms
- Subprograms can be reused in multiple applications

PL/SQL Main Features

- PL/SQL is a procedural language
- PL/SQL supports the declaration of
  - Constants
  - Variables
- Using PL/SQL, a developer can
  - Control program flow
  - Define subprograms
  - Handle runtime errors

PL/SQL Main Features

- PL/SQL provides:
  - Blocks
  - Variables and Constants
  - Error Handling
  - Subprograms
  - Packages
  - Triggers
  - Input and Output
  - Data Abstraction
  - Control Statements
  - Conditional Compilation
  - Processing a Query Result Set One Row at a Time

PL/SQL Main Features Blocks

- The basic unit of a PL/SQL source program is the block
- The block groups related declarations and statements.
- A PL/SQL block is defined by the keywords:
  - DECLARE - declarative part
  - BEGIN - an executable part
  - EXCEPTION - an exception-handling part
  - END.
- Only the executable part is required (BEGIN ... END)
- A block can have a label.

PL/SQL Main Features Blocks

```
<< label >> (optional)
DECLARE    -- Declarative part (optional)
  -- Declarations of local types, variables, & subprograms

BEGIN      -- Executable part (required)
  -- Statements (which can use items declared in declarative part)

[EXCEPTION -- Exception-handling part (optional)
  -- Exception handlers for exceptions (errors) raised in executable part]
END;
```


PL/SQL Main Features Blocks

- Between DECLARE and BEGIN
  - Declaration part of the block
  - A developer can declare here:
    - Variables
    - Cursors
    - User-defined data types
    - User-defined exceptions
    - Subprograms (Functions, Procedures)
  - This part is optional

PL/SQL Main Features
Blocks

- Between BEGIN and EXCEPTION
  - Executable part of the block
  - Contains
    - Executable statements
    - SQL queries
    - Control structures
    - Loop structures
  - The EXECEPTION part of the block is optional
    - In this case, the block ends with the key-word END.

PL/SQL Main Features
Blocks

- Between EXCEPTION and END
  - Exception handler part of the block
  - This part is optional

PL/SQL Main Features
Blocks

- Block example:

```
BEGIN
   null;
END;
/
```


PL/SQL Main Features
Blocks

- There should be at least on instruction in the execution part of the block (BEGIN … END)
- Each instruction in a PL/SQL block ends with a semicolon (;)
- There is no semicolon after DECLARE, BEGIN and EXCEPTION
- There is a semicolon after END
- To run the block in SQL*Plus use:
  - Slash (/)
  - Run (r)

Examples

```
SET AUTOCOMMIT OFF

BEGIN
  INSERT INTO countries
    VALUES('RO', 'Romania', 1);
  INSERT INTO locations(location_id, city, country_id)
    VALUES(4000, 'Bucuresti', 'RO');
  INSERT INTO departments(department_id, department_name, location_id)
    VALUES(500, 'IT Bucuresti', 4000);
  COMMIT;
END;
/
```


Examples

```
SELECT * FROM countries
  WHERE country_id='RO';

SELECT * FROM locations
  WHERE location_id=4000;

SELECT * FROM departments
  WHERE department_id=500;
```


PL/SQL Main Features
Blocks

- Declarations are local to the block
- Declarations cease to exist when the block completes execution
  - This helps to avoid cluttered namespaces for variables and subprograms
- Blocks can be nested:
  - A block is an executable statement
  - A block can appear in another block wherever an executable statement is allowed.

PL/SQL Main Features
PL/SQL Nested Blocks

```
[<<label>>] -- block label (optional)
[DECLARE] -- optional
  -- variables for all the blocks
BEGIN -- start of parent block
    [DECLARE] -- optional
      -- variables for child_1 block
    BEGIN -- start of child_1 block
      -- can access variables from parent and child_1 blocks
    [EXEPTION] -- optional
      -- exception handler for child_1 block
    END; -- end of child_1 block

    [DECLARE] -- optional
      -- variables for child_2 block
    BEGIN -- start of child_2 block
      -- can access variables from parent and child_2 blocks
    [EXCEPTION] -- optional
      -- exception handler for child_2 block
    END; -- end of child_2 block
[EXCEPTION]  -- optional
  -- exception handler for parent block and
  -- any unhandled exception from the child_1 or child_2 blocks
END;
/
```

PL/SQL Main Features
PL/SQL Nested Blocks

```
[<<label>>] -- block label (optional)
[DECLARE] -- optional
   -- variables for all the blocks
BEGIN -- start of parent block

    [DECLARE] -- optional
       -- variables for child_1 block
    BEGIN -- start of child_1 block
       -- can access variables from parent and child_1 blocks

        [DECLARE] -- optional
           -- variables for child_2 block
        BEGIN -- start of child_2 block
           -- can access variables from parent, child_1 and child_2 blocks

            [DECLARE] -- optional
               -- variables for child_3 block
            BEGIN -- start of child_3 block
               -- can access variables from parent and child_3 blocks
            [EXCEPTION] -- optional
               -- exception handler for child_3 block
            END; -- end of child_3 block

        [EXCEPTION] -- optional
           -- exception handler for child_2 block
           -- handler unhandler exception from child_3 block
        END; -- end of child_2 block

    [EXCEPTION] -- optional
       -- exception handler for child_1 block
       -- handler unhandler exception from child_2 and child_3 blocks
    END; -- end of child_1 block

[EXCEPTION]  -- optional
   -- exception handler for parent block and
   -- handler unhandled exception from child_1, child_2 and child_3 blocks
END;
/
```


PL/SQL Main Features
Blocks

- A block can be
  - Submitted to an interactive tool (such as SQL*Plus or Enterprise Manager)
  - Embedded in an Oracle Precompile
  - Embedded in an OCI (Oracle Call Interface) program
- The interactive tool or program runs the block one time

PL/SQL Main Features
Blocks

- The block is not stored in the database
- It is called an anonymous block (even if it has a label)
- An anonymous block is compiled each time it is loaded into memory
- An anonymous block compilation has three stages:
  1. Syntax checking: PL/SQL syntax is checked, and a parse tree is generated.
  2. Semantic checking: Type checking and further processing on the parse tree.
  3. Code generation

Examples

```
BEGIN
  UPDATE locations
    SET postal_code = '252525',
    state_province = 'Sector 7'
    WHERE location_id = 4000;
  COMMIT;
END;
/
```


PL/SQL Main Features
Variables and Constants

- In the DECLARATION part of a PL/SQL block there can be declared
  - Variables
  - Constants
- They can be use them wherever you can use an expression
- As the program runs
  - The values of variables can change
  - The values of constants cannot

PL/SQL Main Features
Variables and Constants

- Example

```
DECLARE
   A NUMBER(5,2);
   PI CONSTANT NUMBER(3,2) := 3.14;
BEGIN
   A := PI * &r ** 2;
END;
/
```


PL/SQL Main Features
Error Handling

- PL/SQL supports error handlers to efficiently detect errors
- When an error occurs:
  - An exception is raised
  - Normal execution stops
  - The control transfers to the exception-handling part of the PL/SQL block.

PL/SQL Main Features
Subprograms

- A PL/SQL subprogram is a named PL/SQL block that can be invoked repeatedly
- If the subprogram has parameters, their values can differ for each invocation
- PL/SQL has two types of subprograms
  1. Procedures – does not return a result
  2. Functions – returns a result
- PL/SQL also lets you invoke external programs written in other languages

PL/SQL Main Features
Packages

- A PL/SQL subprogram is a named PL/SQL block that can be invoked repeatedly
- A package is a schema object that groups logically related PL/SQL
  - Types
  - Variables
  - Constants
  - Subprograms
  - Cursors
  - Exceptions

PL/SQL Main Features
Packages

- A package is compiled and stored in the database
- Applications can share package
- Developers can write their own packages
- Oracle provides many product-specific packages

PL/SQL Main Features
Triggers

- A trigger
  - Is a named PL/SQL unit
  - Is stored in the database
  - Runs in response to an event that occurs in the database

PL/SQL Main Features
Triggers

- A developer can specify
  - The event to be handled by the trigger
  - Whether the trigger fires
    - Before the event
    - After the event
  - Whether the trigger runs for each:
    - Event
    - Row affected by the event

PL/SQL Main Features
Input and Output

- Most PL/SQL input and output (I/O) is done with SQL statements
- These statements
  - Store data in database tables
  - Query those tables
- PL/SQL I/O is also done with PL/SQL packages that Oracle Database supplies

PL/SQL Main Features
Input and Output

Package	Description	More Information
DBMS_OUTPUT	Lets PL/SQL blocks, subprograms, packages, and triggers display output. Especially useful for displaying PL/SQL debugging information.	*Oracle Database PL/SQL Packages and Types Reference*
HTF	Has hypertext functions that generate HTML tags (for example, the HTF.ANCHOR function generates the HTML anchor tag <A>).	*Oracle Database PL/SQL Packages and Types Reference*
HTP	Has hypertext procedures that generate HTML tags.	*Oracle Database PL/SQL Packages and Types Reference*
DBMS_PIPE	Lets two or more sessions in the same instance communicate.	*Oracle Database PL/SQL Packages and Types Reference*
UTL_FILE	Lets PL/SQL programs read and write operating system files.	*Oracle Database PL/SQL Packages and Types Reference*
UTL_HTTP	Lets PL/SQL programs make Hypertext Transfer Protocol (HTTP) callouts, and access data on the Internet over HTTP.	*Oracle Database PL/SQL Packages and Types Reference*
UTL_SMTP	Sends electronic mails (emails) over Simple Mail Transfer Protocol (SMTP) as specified by RFC821.	*Oracle Database PL/SQL Packages and Types Reference*

PL/SQL Main Features
Data Abstraction

- A developer can specify their own data types
- Data structures can be used to achieve data abstraction
- Developers can use:
  - Cursors
  - Composite Variables
  - The %ROWTYPE Attribute
  - The %TYPE Attribute
  - Abstract Data Types

PL/SQL Main Features
Data Abstraction

- A cursor
  - Is a pointer to a private SQL area that stores information about processing
    - a specific SQL statement
    - PL/SQL SELECT INTO statement
  - Is used to retrieve the rows of the result set one at a time.
- Cursor attributes can be used to get information about the state of the cursor (e.g., how many rows the statement has affected so far)

PL/SQL Main Features
Data Abstraction

- A composite variable has internal components
- An internal component can be accessed individually
- Entire composite variables can be passed to subprograms as parameters

PL/SQL Main Features
Data Abstraction

- PL/SQL has two kinds of composite variables:
  - Collections
    - The internal components (elements) are always of the same data type
    - Each element is accessed by its unique index.
    - E.g., lists and arrays
  - Records
    - The internal components (fields) can be of different data types
    - Each field can be accessed by its name
    - A record variable can hold
      - A table row
      - Some columns from a table row.

PL/SQL Main Features
Data Abstraction

- The %ROWTYPE attribute
  - Is used to declare a record that represents either
    - A full row of a database table or view
    - Partial row of a database table or view
- The %TYPE attribute
  - Is used to declare a data item of the same data type as a previously declared variable or column

Examples

```
1  SET SERVEROUTPUT ON
2
3  DECLARE
4    dn departments.department_name%TYPE;
5    idd departments.department_id%TYPE := 500;
6  BEGIN
7    SELECT department_name INTO dn
8      FROM departments
9      WHERE department_id = idd;
10   DBMS_OUTPUT.PUT_LINE('Department id:   ' || idd);
11   DBMS_OUTPUT.PUT_LINE('Department name: ' || dn);
12 END;
13 /
```


Examples

```
1  SET SERVEROUTPUT ON
2
3  DECLARE
4    dn departments.department_name%TYPE;
5    idd departments.department_id%TYPE := &id;
6  BEGIN
7    SELECT department_name INTO dn
8      FROM departments
9      WHERE department_id = idd;
10   DBMS_OUTPUT.PUT_LINE('Department id:   ' || idd);
11   DBMS_OUTPUT.PUT_LINE('Department name: ' || dn);
12 END;
13 /
```


Examples

```
1  SET SERVEROUTPUT ON
2
3  DECLARE
4    dn departments.department_name%TYPE;
5  BEGIN
6    SELECT department_name INTO dn
7      FROM departments
8      WHERE department_id = &&idd;
9    DBMS_OUTPUT.PUT_LINE('Department id:   '||&idd);
10   DBMS_OUTPUT.PUT_LINE('Department name: '||dn);
11 END;
12 /
13
14 define
15 undef idd
```


Examples

```
1  SET SERVEROUTPUT ON
2
3  DECLARE
4    country countries%ROWTYPE;
5  BEGIN
6    SELECT * INTO country
7      FROM countries
8      WHERE country_id = &id;
9    DBMS_OUTPUT.PUT_LINE('Country id:   '|| country.country_id);
10   DBMS_OUTPUT.PUT_LINE('Country name: '|| country.country_name);
11 END;
12 /
```


PL/SQL Main Features
Data Abstraction

- An Abstract Data Type (ADT) consists of a data structure and subprograms that manipulate the data
- The variables that form the data structure are called attributes
- The subprograms that manipulate the attributes are called methods
- ADTs are stored in the database
- Instances of ADTs can be
  - Stored in tables
  - Used as PL/SQL variables.

PL/SQL Main Features
Data Abstraction

- ADTs reduce complexity by separating a large system into logical reusable components
- ADTs are created using CREATE TYPE Statement.
- ADTs are also called user-defined types and object types

PL/SQL Main Features
Conditional Compilation

- Conditional compilation enables developers to customize the functionality in a PL/SQL application without removing source text
- For example:
  - Use new features with the latest database release
  - Disable new features when running the application in an older database release
  - Activate debugging or tracing statements in the development environment
  - Hide debugging or tracing statements when running the application at a production site

PL/SQL Main Features
Processing a Query Result Set One Row at a Time

- Inside PL/SQL, the results of a SQL query can be processed one row a time
- This can be achieved by using:
  - A basic loop
  - An individual statements to
    - Run the query
    - Retrieve the results
    - Finish processing

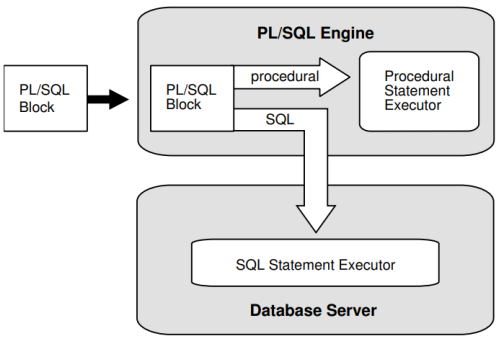
Overview

- PL/SQL
- PL/SQL Main Features
- PL/SQL Engine

PL/SQL Engine

- The PL/SQL compilation and runtime system is an engine
- The PL/SQL engine compiles and runs PL/SQL units
- The PL/SQL engine can be installed:
  - In the database
  - In an application development tool (e.g., Oracle Forms)
- Regardless of the environment, the PL/SQL engine accepts as input any valid PL/SQL unit

PL/SQL Engine



PL/SQL Engine

- The engine
  - Runs procedural statements
  - Sends SQL statements to the SQL engine in the database
  - PL/SQL units are processed by:
    - The database if the database PL/SQL engine is used
    - The local PL/SQL engine if an application development tool is used
  - If a PL/SQL unit contains no SQL statements
    - The local engine processes the entire PL/SQL unit
    - This is useful if the application development tool can benefit from conditional and iterative control

PL/SQL Engine

- Why using PL/SQL Engine is useful:
  - Oracle Forms applications frequently use SQL statements to
    - Test the values of field entries
    - Do simple computations
  - By using PL/SQL instead of SQL
    - Avoid calls to the database

Examples

```
DECLARE
  country countries%ROWTYPE;
  region regions%ROWTYPE;
BEGIN
  SELECT
    c.country_id,
    c.country_name,
    r.region_id,
    r.region_name
  INTO
    country.country_id,
    country.country_name,
    region.region_id,
    region.region_name
  FROM countries c
    INNER JOIN regions r
      on c.region_id = r.region_id
    WHERE country_id = &id;
  DBMS_OUTPUT.PUT_LINE('Country id:   ' || country.country_id);
  DBMS_OUTPUT.PUT_LINE('Country name: ' || country.country_name);
  DBMS_OUTPUT.PUT_LINE('Region id:    ' || region.region_id);
  DBMS_OUTPUT.PUT_LINE('Region name:  ' || region.region_name);
END;
/
```


Examples

```
BEGIN
  DELETE FROM departments
    WHERE department_id = 500;
  DELETE FROM locations
    WHERE location_id = 4000;
  DELETE FROM countries
    WHERE country_id = 'RO';
  COMMIT;
END;
/
```


Variables

```
v_valoare NUMBER(15) NOT NULL := 0;
v_data_achizitie DATE DEFAULT SYSDATE;
v_material VARCHAR2(15) := 'Matase';
c_valoare CONSTANT NUMBER := 100000;
v_stare VARCHAR2(20) DEFAULT 'Buna';
v_clasificare BOOLEAN DEFAULT FALSE;
int_an_luna INTERVAL YEAR TO MONTH := INTERVAL '3-2' YEAR TO
MONTH;
```

variable_name [CONSTANT] datatype [NOT NULL] [:= | DEFAULT initial_value]

- When declaring a **variable**, **storage space** is allocated for a value of a specified **data type**
- The **storage location** is **marked** with a name so it can be **referenced**
- Objects must be declared **before** referencing them
- Declarations can appear in the **declarative** part of any Block / Subprogram / Package

Datatypes:

- **Scalar**: Contain a **single** value (No internal components)
- **Composit**: **Collections** / **Records**

variable_name [CONSTANT] datatype [NOT NULL] [:= | DEFAULT initial_value]

- **variable_name** - Name of the variable that you are declaring
- **CONSTANT –** Imposes that the value cannot be changed after assignment
- **datatype** - Name of a scalar data type, including any qualifiers for size, precision, and character or byte semantics
- **NOT NULL** - Imposes the NOT NULL constraint on the variable
- **:= / DEFAULT –** Assigning the initial value
- **initial_value** – Immediate value or expression

Variables

```
v_valoare NUMBER(15) NOT NULL := 0;
v_data_achizitie DATE DEFAULT SYSDATE;
v_material VARCHAR2(15) := 'Matase';
c_valoare CONSTANT NUMBER := 100000;
v_stare VARCHAR2(20) DEFAULT 'Buna';
v_clasificare BOOLEAN DEFAULT FALSE;
int_an_luna INTERVAL YEAR TO MONTH := INTERVAL '3-2' YEAR TO
MONTH;
```

NOT NULL Constraint

An item can acquire this **constraint** either **Implicitly** (from its data type) or **Explicitly**

The **default initial value** for a scalar variable is NULL.

A scalar variable declaration that specifies **NOT NULL** (either implicitly or explicitly) **must** assign **an initial value** to the variable.

PL/SQL treats any **zero-length string** as a **NULL** value, this includes:

- Values returned by character functions
- BOOLEAN expressions.

Initial Values of Variables and Constants

If the declaration is

- **A variable**: the initial value is optional unless the NOT NULL constraint is specified
- **A constant**: the initial value is required

If the declaration is in a **block** or **subprogram**:

- The initial value is assigned to the variable or constant **every time** control passes to the block or subprogram

If the declaration is in a **package specification**

- The initial value is assigned to the variable or constant for **each session** (whether the variable or constant is public or private)

Initial Values of Variables and Constants

To specify the **initial value** use

- The assignment operator (**:=**) followed by an expression
- The keyword **DEFAULT** followed by an expression

The expression can include **previously declared**

- **Constants**
- **Initialized variables**
- **Functions results**

If **no initial value** is specified for a variable, then a value **must be assigned before using it** in any other context.

Example

```
DECLARE
    no_months constant BINARY_INTEGER:= 12;
    emp_name varchar2(45);
    emp_salary employees.salary%TYPE;
    anual_salary float(10);
BEGIN
    SELECT first_name || ' ' || last_name, salary
    INTO emp_name, emp_salary
    FROM employees WHERE employee_id = &id;

    anual_salary := emp_salary * no_months;
    dbms_output.put_line(rpad(emp_name, 20) || '-' || lpad(anual_salary, 10));
END;
/
```

Initial Values of Variables and Constants

Every PL/SQL **constant**, **variable**, **parameter**, and **function** (block that returns a value) has a data type.

The data type determines

- The storage format
- Valid values
- Operations

The data type can be assiged

- **Explicit** – by naming the data type used (e.g. **NUMBER**, **VARCHAR2**, etc.)
- **Implicit** – by using attributes (**%TYPE**, **%ROWTYPE**)

Declaring **Variables** using **%TYPE**

The **%TYPE** attribute permits the declaration of a data item of the same data type as a previously declared **variable** or **column** (**without** knowing what that type is).

If the declaration of the **referenced** item changes, then the declaration of the **referencing** item changes accordingly.

The syntax of the declaration is: referencing_item referenced_item**%TYPE**;

Declaring **Variables** using **%TYPE**

The referencing item **inherits** the following from the **referenced** item:

- **Data type** and **size**
- **Constraints** (unless the referenced item is a column)

The referencing item **does not inherit** the initial **value** of the referenced item: If the referencing item specifies or inherits the **NOT NULL** constraint, an initial **value** for it **must be specified**.

The **%TYPE** attribute is particularly useful when declaring variables to hold **database values.**

Example

```
DECLARE
    a INT NOT NULL := 4;
    b a%TYPE := 3;

    c employees.employee_id%TYPE := 4;
    d c%TYPE default 3;
BEGIN
    dbms_output.put_line(a);
    dbms_output.put_line(b);

    dbms_output.put_line(c);
    dbms_output.put_line(d);
END;
/
```

Scalar data types

- Scalar data types store values with **no internal components**
- A scalar data type can have **subtypes**
- A **subtype** is a data type that is a **subset** of another data type (its base type)
- A **subtype** has the same valid **operations** as its base type
- A data type and its subtypes comprise a **data type family**
- PL/SQL **predefines** many types and subtypes
- The **predefined** data types are available in the **STANDARD package**
- Developers can also define **their own subtypes** (**UDST** – User Defined Subtypes)

Scalar data types

The PL/SQL **scalar** data types are:

- The SQL data types
- BOOLEAN
- PLS_INTEGER
- BINARY_INTEGER
- REF CURSOR
- User-defined subtypes

SQL data types

- **Character** Data Types
- **Number** Data Types
- **Datetime** and **Interval** Data Types
- **RAW** and **LONG RAW** Data Types
- Large Object (**LOB**) Data Types
- **Rowid** Data Types

Character Data Types

CHAR : specifies a **fixed**-length character string in the database character set

VARCHAR2 : specifies a **variable**-length character string in the database character set.

ASCII VS **Unicode**

NCHAR : specifies a **fixed**-length character string in the national character set

NVARCHAR2 : specifies a **variable**-length character string in the national character set.

Number Data Types

The NUMBER data type stores zero as well as positive and negative fixed **numbers**.

NUMBER(p, s)

- p is the **precision**: the **maximum** number of **significant decimal digits**
- s is the **scale**: the number of digits from the **decimal point** to the least significant digit.

Number Data Types

The **FLOAT** data type is a **subtype** of **NUMBER**. It can be specified with or without **precision**. Scale cannot be specified, is interpreted from the data.

FLOAT(p) - p is the precision: the maximum number of **significant decimal digits**

Floating-point numbers can have a decimal point anywhere from the first to the last digit or can have no decimal point at all.

Oracle Database provides two numeric data types exclusively for floating-point numbers:

- **BINARY_FLOAT** is a **32-bit**, single-precision floating-point number data type. Each **BINARY_FLOAT** value requires **4 bytes**.
- **BINARY_DOUBLE** is a **64-bit**, double-precision floating-point number data type. Each **BINARY_DOUBLE** value requires **8 bytes**.

FLOAT vs **BINARY_FLOAT** vs **BINARY_DOUBLE**

- **BINARY_FLOAT** & **BINARY_DOUBLE** data types take advantage of **hardware acceleration**, therefore, they have better performance for numerical computations.
- **BINARY_FLOAT** & **BINARY_DOUBLE** data types can store smaller / larger numbers than **FLOAT** type.
- **BINARY_FLOAT** & **BINARY_DOUBLE** data types store only **approximate** values, while the **FLOAT** data type stores **exact** values.
- **BINARY_FLOAT** & **BINARY_DOUBLE** data types are suitable for the **scientific calculations** but **not** suitable for **financial calculations**.

Datetime and **Interval** Data Types

The **DATE** data type stores **date** and **time** information

- Oracle stores the following information for a **DATE**: year, month, day, hour, minute, and second

The **TIMESTAMP** data type is an **extension** of the **DATE** data type. It stores all information from of the **DATE** data type plus **fraction of seconds**. This data type is useful for

- Storing **precise** time values
- **Collecting** and **Evaluating** date information across **geographic** regions

timestamp_var **TIMESTAMP** [(fractional_seconds_precision)]

timestamp_with_local_var **TIMESTAMP** [(fractional_seconds_precision)] **WITH LOCAL TIME ZONE**

- **TIMESTAMP WITH LOCAL TIME ZONE** is another variant of **TIMESTAMP** that is sensitive to time zone information.
- **fractional_seconds_precision** optionally specifies the **number of digits** Oracle stores in the fractional part of the SECOND datetime field.

Datetime and **Interval** Data Types

INTERVAL YEAR TO MONTH stores a period of time using the **YEAR** and **MONTH** datetime fields. This data type is useful for representing the difference between two datetime values when only the year and month values are significant.

INTERVAL YEAR [(year_precision)] **TO MONTH**

- **year_precision** is the number of **digits** in the **YEAR** datetime field

Datetime and **Interval** Data Types

INTERVAL DAY TO SECOND stores a period of time in terms of days and seconds. This data type is useful for representing the precise difference between two datetime values

INTERVAL DAY [(day_precision)] **TO SECOND** [(fractional_seconds_precision)]

- **day_precision** is the number of digits in the **DAY** datetime field. Accepted values are 0 to 9. The default is **2**.
- **fractional_seconds_precision** is the number of digits in the fractional part of the **SECOND** datetime field. Accepted values are 0 to 9. The default is **6**.

RAW and **LONG RAW** Data Types

The **RAW** and **LONG RAW** data types store data that is **not to be explicitly converted** by Oracle Database when moving data between different systems. These data types are intended for **binary data** or **byte strings**. Oracle **strongly recommends** that you convert **LONG RAW** columns to binary **LOB** (BLOB) columns. **LOB** columns are subject to far **fewer restrictions** than **RAW** columns. The built-in LOB data types:

- BLOB
- CLOB
- NCLOB (stored internally)
- BFILE (stored externally)

LOBs can store **large** and **unstructured** data such as text, image, video, and spatial data.

BLOB vs **CLOB** vs **NCLOB** Data Types

The **BLOB** data type stores **unstructured binary large objects**. The **CLOB** data type stores **single-byte** (ASCII) character data or **multibyte** (Unicode) character data. The **NCLOB** data type stores **Unicode** data.

BLOB objects can be thought of as bitstreams with no character set semantics.

CLOB and **NCLOB** support:

- **Fixed-width** character sets (uses the database character set)
- **Variable-width** character sets (uses the database character set)

ALL of Them store binary data up to (4 GB - 1B) * LOB_CHUNK_SIZE

- **LOB_CHUNK_SIZE** is **equal** to the database **block size**

BFILE Data Types

The **BFILE** data type enables access to binary file **LOBs** that are stored in file systems **outside** Oracle Database. A **BFILE column** or **attribute** stores a **BFILE locator**.

BFILE locator serves as a **pointer** to a **binary file** on the **server file system**. The **locator** maintains the **directory name** and the **filename**.

Binary file **LOBs** do not participate in transactions and are **not recoverable**. The **operating system** provides file **integrity** and **durability**.

BFILE data can be up to 2^64-1 bytes (the operating system may impose restrictions on this maximum). The **database administrator** must **ensure** that the **external** file exists. **Oracle processes** have operating system **read** permissions on the file.

The **BFILE** data type enables **read-only** support of **large binary files**, such a file cannot be modified or replicated.

ROWID Data Types

Each row in the database has an **address**. The rows in **heap-organized tables** that are native to Oracle Database have **row addresses** called **rowids**.

A **rowid** row address can be examined by querying the **pseudocolumn ROWID**. Values of this **pseudocolumn** are **strings** representing the **address of each row**. These strings have the data type **ROWID**. **Rowids** contain the following information:

- The data block of the data file containing the row.
- The row in the data block.
- The database file containing the row
- The data object number which is an identification number assigned to every database segment.

BOOLEAN Data Type

The PL/SQL data type **BOOLEAN** stores logical values: **TRUE** / **FALSE** / **NULL** (representing an unknown value)

Because SQL has **no** data type **equivalent** to **BOOLEAN**, it is **not** possible to:

- Assign a **BOOLEAN** value to a database **table column**
- **Select** or **fetch** the value of a database table column **into a BOOLEAN** variable
- Use a **BOOLEAN** value in a **SQL function**
- Use a **BOOLEAN** expression in a **SQL statement**, except
  - As an **argument** to a PL/SQL **function** invoked in a SQL query
  - In a PL/SQL **anonymous block**

PLS_INTEGER vs **BINARY_INTEGER** Data Types

The PL/SQL data types **PLS_INTEGER** and **BINARY_INTEGER** are identical. Both data types:

- Store **signed** integers in the range -2,147,483,648 through 2,147,483,647
- **Integers** are represented in **32 bits**

They have the following advantages over the **NUMBER** data type and **NUMBER subtype**s:

- Values require **less storage**
- Operations use **hardware arithmetic**
- Are faster than NUMBER operations which use **library arithmetic**

User-Defined PL/SQL Subtypes

Developers can define their **own subtypes** for restrincting: **Size** / **Precision** / **Scale**

```
SUBTYPE subtype_name IS base_type
  { precision [, scale ] | RANGE low_value .. high_value } [ NOT NULL ]
```

Basically **syntactic sugar** for some types. (see **INT** vs **NUMBER**)

Supported Operators in PL/SQL

- Arithmetic operators
- Logic operators
- Comparison operators
- SQL operators

Arithmetic operators

Operator	Operation	Example
**	***Exponential***	val2 ** val1
+	***Identity***	+ val
-	***Negation***	- val
+	***Addition***	val1 + val2
-	***Subtraction***	val1 - val2
*	***Multiplication***	val1 * val2
/	***Division***	val1 / val2
||	***Concatenation***	val1 || val2

Logical operators

Operator	Operation	Example
NOT	***Negation***	NOT val
AND	***Logical AND***	val1 AND val2
OR	***Logical OR***	val1 OR val2

Comparison operators

OPERATOR	OPERATION	EXAMPLE
=	***Equal***	val2 = val1
<	***Less then***	val2 < val1
>	***Greater than***	val2 > val1
<=	***Less or equal than***	val1 <= val2
>=	***Greater or equal than***	val1 >= val2
<>	***Different than***	val1 <> val2
!=	***Different than***	val1 != val2
~=	***Different than***	val1 ~= val2
^=	***Different than***	val1 ^= val2

SQL operators

OPERATOR	OPERATION	EXAMPLE
BETWEEN	***Verifies if a value is in a range***	val BETWEEN val1 AND val2
IN	***Verifies if a value is in a list***	val IN(val1, val2, …, val3)
LIKE	***Verifies a specified pattern***	val LIKE pattern
IS NULL	***Verifies if a value is NULL***	val IS NULL

Operators precedence

PRECEDENCE	OPERATORS	OPERATION
1	**	Exponential
2	+, -	Identity, negation
3	*, /	Multiplication, division
4	+, -, ||	Addition, subtraction, concatenation
5	=, <, >, <=, >=, <>, !=, ~=, ^=, IS NULL, LIKE, BETWEEN, IN.	Comparison and SQL operators
6	NOT	Logical negation
7	AND	Conjunction
8	OR	Inclusion

Other symbols

Symbol	Meaning
:=	Assignment operator
=>	Association operator
%	Attribute indicator
'	String delimiter
.	Component selector
( )	Expression or list delimiter
:	Host variable indicator
,	Item separator
<< >>	Label delimiter
/* */	Multiline comment delimiter
"	Quoted identifier delimiter
..	Range operator
@	Remote access indicator
--	Single-line comment indicator
;	Statement terminator

Control Statements

- Conditional selection statements
- Loop statements
- Sequential control statements

Conditional selection statements

IF Statement

The condition can contain:

- Comparison operators
- Logical operators
- SQL operators

```
-- IF THEN Statement
IF condition THEN statements
END IF;

-- IF THEN ELSE Statement
IF condition THEN statements
ELSE else_statements
END IF;

-- IF THEN ELSIF Statement
IF condition_1 THEN statements_1
ELSIF condition_2 THEN statements_2
[ ELSIF condition_3 THEN statements_3 ]
...
[ ELSE else_statements ]
END IF;

-- Logically equivalent nested
-- IF THEN ELSE statements
IF condition_1 THEN statements_1;
ELSE
  IF condition_2 THEN statements_2;
  ELSE
    IF condition_3 THEN statements_3;
    ...
    END IF;
  END IF;
END IF;
```

```
DECLARE
    emp_name varchar2(45);
    emp_salary employees.salary%TYPE;
    emp_iddept employees.department_id%TYPE;
    avg_sal number(6,2);
    avg_sal_dept float(8);
BEGIN
    SELECT first_name ||' '|| last_name, salary, department_id
    INTO emp_name, emp_salary, emp_iddept
    FROM employees WHERE employee_id = &id;

    SELECT AVG(salary) INTO avg_sal FROM employees;

    SELECT AVG(salary) INTO avg_sal_dept FROM employees
    WHERE department_id= emp_iddept;

    IF emp_salary > avg_sal THEN
        dbms_output.put_line(emp_name || ' are sal > decat sal mediu din firma');
    END IF;

    IF emp_salary > avg_sal_dept THEN
        dbms_output.put_line(emp_name || ' are sal > sal mediu din departamentul in care lucreaza');
    ELSE
        dbms_output.put_line(emp_name || ' are sal <= sal mediu din departamentul in care lucreaza');
    END IF;
END;
/
```

```
DECLARE
    year_of_experience INTERVAL YEAR TO MONTH;
    expert INTERVAL YEAR TO MONTH;
    emp_name varchar2(45);
    emp_hiredate employees.hire_date%TYPE;
BEGIN
    expert := INTERVAL '15-1' YEAR TO MONTH;

    dbms_output.put_line(expert);
    dbms_output.put_line(sysdate());
    dbms_output.put_line(sysdate - expert);
    SELECT first_name ||' '|| last_name, hire_date
    INTO emp_name, emp_hiredate
    FROM employees WHERE employee_id = &id;

    IF emp_hiredate < sysdate - expert THEN
        dbms_output.put_line(emp_name || ' e expert');
    ELSE
        dbms_output.put_line(emp_name || ' nu e expert');
    END IF;
END;
/
```

Conditional selection statements

CASE Statement

Simple CASE

- The selector is an expression (typically a single variable)
- Each selector_value can be either a literal or an expression (cannot use NULL)
- The simple CASE statement runs the first statements for which selector_value equals selector
- Remaining conditions are not evaluated
- If no selector_value equals selector the CASE statement runs else_statements if they exist
- Raises the predefined exception CASE_NOT_FOUND otherwise

Searched CASE

- The searched CASE statement runs the first statements for which condition is true
- Remaining conditions are not evaluated

```
-- Simple CASE
CASE selector
WHEN selector_value_1 THEN statements_1
WHEN selector_value_2 THEN statements_2
...
WHEN selector_value_n THEN statements_n
[ ELSE else_statements ]
END CASE;

-- Searched CASE
CASE
WHEN condition_1 THEN statements_1
WHEN condition_2 THEN statements_2
...
WHEN condition_n THEN statements_n
[ ELSE else_statements ]
END CASE;
```

```
DECLARE
    emp_name varchar2(45);
    emp_iddept employees.department_id%TYPE;
    emp_reg regions.region_name%TYPE;
    emp_city locations.city%TYPE;
BEGIN
    SELECT first_name ||' '|| last_name, department_id
    INTO emp_name, emp_iddept
    FROM employees WHERE employee_id = &id;

    SELECT r.region_name, l.city INTO emp_reg, emp_city FROM regions r
        INNER JOIN countries c ON c.region_id = r.region_id
        INNER JOIN locations l ON l.country_id = c.country_id
        INNER JOIN departments d ON d.location_id = l.location_id
    WHERE d.department_id = emp_iddept;

    CASE emp_reg
        WHEN 'Europe' THEN
            IF emp_city = 'London' THEN
                dbms_output.put_line(emp_name || ' works in European HQ');
            ELSE
                dbms_output.put_line(emp_name || ' doesn''t works in HQ');
            END IF;
        WHEN 'Americas' THEN
            IF emp_city = 'Seattle' THEN
                dbms_output.put_line(emp_name || ' works in American HQ');
            ELSE
                dbms_output.put_line(emp_name || ' doesn''t works in HQ');
            END IF;
        ELSE
            dbms_output.put_line(emp_name || ' works outside Europe and America');
    END CASE;
END;
/
```

```
DECLARE
    emp_name varchar2(45);
    comm employees.commission_pct%TYPE;
BEGIN
    SELECT first_name ||' '|| last_name, commission_pct
    INTO emp_name, comm
    FROM employees WHERE employee_id = &id;

    CASE
        WHEN comm IS NULL THEN
            dbms_output.put_line(emp_name || ' nu are comision');
        WHEN comm >= 0 and comm < 0.2 THEN
            dbms_output.put_line(emp_name || ' are comision mic');
        WHEN comm >= 0.2 and comm < 0.3 THEN
            dbms_output.put_line(emp_name || ' are comision mediu');
        ELSE
            dbms_output.put_line(emp_name || ' are comision mare');
    END CASE;
END;
/
```

```
DECLARE
    emp_name varchar2(45);
    comm employees.commission_pct%TYPE;
BEGIN
    SELECT first_name || ' ' || last_name, commission_pct
    INTO emp_name, comm
    FROM employees WHERE employee_id = &id;

    IF comm IS NULL THEN
        dbms_output.put_line(emp_name || ' nu are comision');
    ELSIF comm >= 0 and comm < 0.2 THEN
        dbms_output.put_line(emp_name || ' are comision mic');
    ELSIF comm >= 0.2 and comm < 0.3 THEN
        dbms_output.put_line(emp_name || ' are comision mediu');
    ELSE
        dbms_output.put_line(emp_name || ' are comision mare');
    END IF;
END;
/
```

Loop Statements

Basic LOOP Statement

With **each iteration** of the loop:
- The statements are run
- Control returns to the top of the loop

To prevent an **infinite loop** and exit the loop:
- Use a statement
- Raised exception

```
-- Basic LOOP
[label] LOOP
  statements
  [ EXIT [ WHEN condition ] ]
  [ CONTINUE [ WHEN condition ] ]
END LOOP [label];
```

EXIT statements

The **EXIT** statement **ends** the **current iteration** of a loop **unconditionally**. It transfers control to the **end** of either the **current loop** or an **enclosing labeled** loop.

The **EXIT WHEN** statement ends the current iteration of a loop when the condition in its **WHEN** clause is **true**. The condition in the **WHEN** clause is evaluated **each time** control reaches the **EXIT WHEN** statement. The **EXIT WHEN** statement does nothing if the condition is not true.

CONTINUE statement

The **CONTINUE** statement **continues** the **current iteration** of a loop **unconditionally**. It transfers control to the **next iteration** of either the **current loop** or an **enclosing labeled** loop.

The **CONTINUE WHEN** statement continues the current iteration of a loop when the condition in its **WHEN** clause is true. The condition in the **WHEN** clause is evaluated each time control reaches the **CONTINUE WHEN** statement. The **CONTINUE WHEN** statement does nothing if the condition is not true.

```
DECLARE
    idx number NOT NULL := 1;
    deptname departments.department_name%TYPE;
    avg_sal_dept number(7,2);
BEGIN
    LOOP
        SELECT d.department_name, avg(e.salary)
        INTO deptname, avg_sal_dept
        FROM departments d
            INNER JOIN employees e
                ON d.department_id = e.department_id
        WHERE d.department_id = idx * 10
        GROUP BY d.department_name;

        dbms_output.put_line(deptname || ' ' || avg_sal_dept);
        idx := idx + 1;

        CONTINUE WHEN idx < 11;

        -- EXIT WHEN idx = 12;

    END LOOP;
END;
/
```

Loop Statements
WHILE LOOP Statement

The **WHILE** LOOP statement runs one or more statements **while** a condition is true.

If the condition is true the statements are run, then **control returns** to the **top of the loop** where condition is **evaluated** again.

The **WHILE** LOOP **stops** when the condition is false. A statement inside the loop must make the condition false or null to prevent an **infinite loop**.

An **EXIT**, **EXIT WHEN**, **CONTINUE**, or **CONTINUE WHEN** in the statements can cause the loop or the current iteration of the loop to end **early**.

```
-- WHILE LOOP
[label] WHILE condition LOOP
  statements
  [ EXIT [ WHEN condition ] ]
  [ CONTINUE [ WHEN condition ] ]
END LOOP [label];
```

```
DECLARE
    idx number := 1;
    deptname departments.department_name%TYPE;
    avg_sal_dept number(7,2);
BEGIN
    WHILE idx < 12
    LOOP
        SELECT d.department_name, avg(e.salary)
        INTO deptname, avg_sal_dept
        FROM departments d
        INNER JOIN employees e
        ON d.department_id = e.department_id
        WHERE d.department_id = idx * 10
        GROUP BY d.department_name;

        dbms_output.put_line(deptname || ' ' || avg_sal_dept);
        idx := idx + 1;
    END LOOP;
END;
/
```

Loop Statements
FOR LOOP Statement

The **FOR** LOOP statement runs one or more statements while the **loop index** is in a **specified range**.

Without **REVERSE**:
- The value of index starts at **lower_bound** and **increases by one** with each iteration of the loop until it reaches **upper_bound**
- The statements **never run if** lower_bound is **greater than** upper_bound

With **REVERSE**:
- The value of index starts at **upper_bound** and **decreases by one** with each iteration of the loop until it reaches **lower_bound**
- If upper_bound is **less than** lower_bound, then the statements **never run**

An **EXIT**, **EXIT WHEN**, **CONTINUE**, or **CONTINUE WHEN** in the statements can cause the loop or the current iteration of the loop to end **early**.

```
-- FOR LOOP
[label] FOR IN [ REVERSE ] LOWER_BOUND .. UPPER_BOUND
LOOP
  statements
  [ EXIT [ WHEN condition ] ]
  [ CONTINUE [ WHEN condition ] ]
END LOOP [label];
```

```
DECLARE
    deptname departments.department_name%TYPE;
    avg_sal_dept number(7,2);
BEGIN
    FOR idx IN 1 .. 12
    LOOP
        dbms_output.put_line('Index =  ' || idx);
        IF idx = 12 THEN
            GOTO exit_for;
        END IF;

        SELECT d.department_name, avg(e.salary)
        INTO deptname, avg_sal_dept
        FROM departments d
            INNER JOIN employees e
                ON d.department_id = e.department_id
        WHERE d.department_id = idx * 10
        GROUP BY d.department_name;

        dbms_output.put_line(deptname || ' ' || avg_sal_dept);
    END LOOP;

    <<exit_for>>dbms_output.put_line('Done!');
END;
/
```

Conditional selection statements: **GOTO**

The sequential control statement **GOTO** transfers control to a label **unconditionally**.

Can be used to **exit** a LOOP. NULL statement only passes control to the next statement

```
GOTO label_example;

<<label_example>>
--Code
```

```
DECLARE
    deptname departments.department_name%TYPE;
    avg_sal_dept number(7,2);
BEGIN
    FOR idx IN 1 .. 12
    LOOP
        dbms_output.put_line('Index =  ' || idx);
        IF idx = 12 THEN
            GOTO exit_for;
        END IF;

        SELECT d.department_name, avg(e.salary)
        INTO deptname, avg_sal_dept
        FROM departments d
            INNER JOIN employees e
                ON d.department_id = e.department_id
        WHERE d.department_id = idx * 10
        GROUP BY d.department_name;

        dbms_output.put_line(deptname || ' ' || avg_sal_dept);
    END LOOP;

    <<exit_for>>dbms_output.put_line('Done!');
END;
/
```

Overview - Composite data types

Records

Collection Types: Associative Arrays, VARRAYS, Nested Tables

Collection Features

Composite data types

A **composite** data type stores values that have **internal components**. PL/SQL lets you define two kinds of composite data types:

- **Record**
- **Collection**

A **composite variable** can be passed to **subprograms** as **parameters** and its **internal components** can be accessed **individually**. Internal components can be either **Scalar** or **Composite**.

Records

The **internal components** of a **record** can have **different** data types and are called **fields**.

Each **field** of a record variable can be accessed by its name, e.g., variable_name.field_name.

Record variables can be:

- **Copies** of existing record variables
  - Created using the attribute **%ROWTYPE** or **%TYPE**
  - Inherits the only **structure**, **not** the **values**
- **User defined**
  - Define a **RECORD** type
  - Create a **variable** of that type

```
DECLARE
    TYPE rec1 IS RECORD(
        id countries.country_id%TYPE,
        name varchar2(20),
        reg regions%ROWTYPE
    );
    r1 rec1;
    r2 r1%type;
BEGIN
    r2.id := 1;
    r2.name = 'Hi, My Name is';
    dbms_output.put_line(r2.id);
END;
/
```

Records

A **RECORD** type defined in a PL/SQL **block** is a **local** type available **only** in the **block.** It is **stored** in the **database** only if the block is in a **subprogram** or **package**.

A **RECORD** type defined in a **package specification** is a **public** item and can be referenced from **outside** the **package** by qualifying it with the package name (package_name.type_name). It is **stored** in the database until you **drop** the **package** with the **DROP PACKAGE** statement.

A **RECORD** type cannot be created at **schema level** (a RECORD type cannot be an ADT attribute data type).

Records

To define a **RECORD** type, specify the record's **name** and define its **fields**. To define a **field**, specify the field's **name** and a its data **type**. The **initial** value of a **field** is **NULL**. The **NOT NULL** constraint can be specified for a field (a non-NULL **initial value** must also be **specified**).

A **non-NULL** initial value is **optional without** the **NOT NULL** constraint.

A **RECORD** type defined in a **package specification** is **incompatible** with an identically defined **local RECORD** type.

A **field** declared with **%ROWTYPE** or **%TYPE** still has an **initial NULL** value. (the variable **does not inherit** the initial **value** of the referenced item)

Records

Records **cannot** be tested natively for **nullity**, **equality**, or **inequality**.

These BOOLEAN expressions are **illegal**:

- My_Record **IS NULL**
- My_Record_1 **=** My_Record_2
- My_Record_1 **>** My_Record_2

Records

Record variables are **allowed** only in these places:

- On the **right side** of the **SET** clause in an **UPDATE** statement (**Only 1** SET clause is allowed in this case). This can be combined with the "**ROW**" keyword on the **left side**.
- In the **VALUES** clause of an **INSERT** statement
- In the **INTO** subclause of a **RETURNING** clause

Record variables are **not allowed** in a **SELECT** list, a **WHERE** clause, a **GROUP BY** clause or in an **ORDER BY** clause. The "**ROW**" keyword **cannot** be used with a **subquery**.

Records

The following **restrictions** apply to **record inserts** and **updates**:

- If the **VALUES** clause of an **INSERT** statement contains a **RECORD** variable, **no other variable** or **value** is allowed in the **clause**
- If the **INTO** subclause of a **RETURNING** clause contains a **RECORD** variable, **no other variable** or **value** is allowed in the **subclause**

These are **not supported**:

- **Nested RECORD** types
- **Functions** that **return** a **RECORD** type
- Record inserts and updates using the **EXECUTE IMMEDIATE** statement (Dynamic SQL)

PL/SQL engine cares about type compatibility so as long as our instructions leverage the proper types, all is good.



Collection types

The **internal components** of a **collection** have the **same** data **type** and are called **elements**. Elements of a **collection** variable can be accessed by its **unique index**, e.g., variable_name(**index**)

Collections variables can be:

- **Copies** of **existing** composite variables (created using the attribute %TYPE - Inherits the only structure, not the values)
- **User defined** – define a collection type and then create a variable of that type

Collection types

Collections have **one dimension**. A **multidimensional** collection can be modeled by using a **collection** whose **elements** are **collections**. Developers can create:

- A **collection** of **records**: a **collection** that has **elements records** and accessing an **item** "variable_name(**index**).field_name"
- A **record** that contains **collections**: a **record** that has **items collections** and accessing a **field** "variable_name.field_name(**index**)"
- A **collection** of **collections**: a **collection** that has items **a collection** and accessing an item "variable_name(**index_1**)(**index_2**)…(**index_n**)"

Collection types

Collection Type	Number of Elements	Index Type	Dense or Sparse	Uninitialized Status	Defined in	ADT Type Attr?
Associative array (or index-by table)	Unspecified	**String** **PLS_INTEGER** (BINARY_INTEGER)	Either	Empty	In PL/SQL block or package	No
VARRAY (variable-size array)	Specified	Integer	Always dense	Null	In PL/SQL block or package or at schema level	Only defined at schema level
Nested table	Unspecified	Integer	Starts dense, can become sparse	Null	In PL/SQL block or package or at schema level	Only defined at schema level

Number of **Elements**, **Index**, **Dense** or **Sparse**

Number of **Elements**?

- **Specified**: the **maximum** number of elements in the **collection**
- **Unspecified**: the **maximum** number of elements in the collection is the **upper limit of the index type**

Dense or **Sparse**?

- A **dense** collection has **no gaps** between **elements**, every element between the **first** and **last** element is **defined** (meaning it has a value, the value can be **NULL** unless the element has a **NOT NULL** constraint).
- A **sparse** collection **has gaps** between elements.

Uninitialized Status

An **empty** collection **exists** but has **no elements**. The **EXTEND** method must be invoked to **add elements** to an **empty** collection.

A **null** collection (also called an **atomically null** collection) **does not exist**. The collection must be initialized to change it from a null collection to an existing collection. The **EXTEND** method cannot be used to initialize a null collection. Initialization:

- Making it empty
- Assigning a non-NULL value to it

Defined in

Same as for **RECORD**: if it's in a **block**, it is **local** / if it's in a **program specification**, it is **public**. Can be **stored** as part of a **subprogram** else it is gone with the session.

A **collection** type **defined** at **schema level** is a standalone type. It is **stored** in the database until you **drop** it with the **DROP TYPE** Statement.

- Can be **created** with the **CREATE** [**OR REPLACE**] **TYPE**
- Can be **modified** using the **ALTER TYPE**

A collection type defined in a package specification is incompatible with an identically defined local or standalone collection type.

ADT Attribute Data Type

To be an **ADT**(Abstract Data Type) **Attribute** Data Type, a collection type must be a **standalone** collection type. Translating Non-PL/SQL Composite Types to PL/SQL Composite Types:

Non-PL/SQL Composite Type	Equivalent PL/SQL Composite Type
Hash table	Associative array
Unordered table	Associative array
Set	Nested table
Bag	Nested table
Array	VARRAY

Associative Arrays

An **associative array** (formerly called **PL/SQL table** or **index-by table**) is a **set** of **key-value** pairs where each **key** is a **unique index**. The **index** is used to locate the associated **value** with the syntax "variable_name(**index**)".

An **associative array** (like a database table) is **empty** (but not null) until it is **populated**. It can **hold** an **unspecified** number of elements (accessible without knowing their positions). Unlike a database table, it **does not** need **disk space** and **cannot** be manipulated with **DML** statements .

Associative Arrays - Indexes

The data type of **index** can be:

- A **string** type (**VARCHAR2**, **VARCHAR**, **STRING**, or **LONG**)
- **PLS_INTEGER/BINARY_INTEGER**

Indexes are **stored** in **sort order**, but are **not created** in **order**. The **sort order** is determined by the initialization parameter **NLS_SORT** and **NLS_COMP** for string types. National Language Support (**NLS**) parameters such as **NLS_SORT**, **NLS_COMP**, and **NLS_DATE_FORMAT** affect associative arrays indexed by **string**.

Associative Arrays

An **associative array** is **appropriate for**:

- A **relatively small** lookup table constructed in **memory** each time the subprogram is invoked or the package that declares it is initialized.
- To pass **collections** to and from the **database server**: Declare formal **subprogram parameters** of **associative array** types > Bind the **host arrays** to the corresponding actual **parameters** (use Oracle Call Interface (**OCI**) or an Oracle Precompiler) > PL/SQL **automatically converts** between host arrays and associative arrays indexed by **PLS_INTEGER** (You **cannot bind** an associative array **indexed** by **VARCHAR**)

Associative Arrays

The **associative array** type **cannot** be **declared** at **schema level** as it is intended for **temporary data storage**. To make an **associative array persistent** declare it in a **package specification** or populate it in the **package body**.

To pass an **associative array** variable as a **parameter** to a standalone **subprogram** the type of that variable must be declared in a **package specification**. Doing so makes the type **available** to the **invoked subprogram** (which declares a formal parameter of that type) and the **invoking subprogram** or anonymous block (which declares and passes the variable of that type).

The **most efficient** way to pass **collections** to and from the database server is to use **associative arrays** with **FORALL** statement and **BULK COLLECT** clause.



```
set serveroutput on
DECLARE
  TYPE rec IS RECORD(
    country_id countries.country_id%TYPE,
    country_name countries.country_name%TYPE,
    region_id countries.region_id%TYPE
  );
  TYPE aa IS TABLE OF rec INDEX BY PLS_INTEGER;

  v aa; -- create a null associative array
  r rec;

  idx PLS_INTEGER;

  TYPE idxs_aa IS TABLE OF PLS_INTEGER INDEX BY PLS_INTEGER;
  idxs idxs_aa;

  i PLS_INTEGER := 1;
BEGIN
  r.country_id := 'UA';
  r.country_name := 'Ukraina';
  r.region_id := 1;

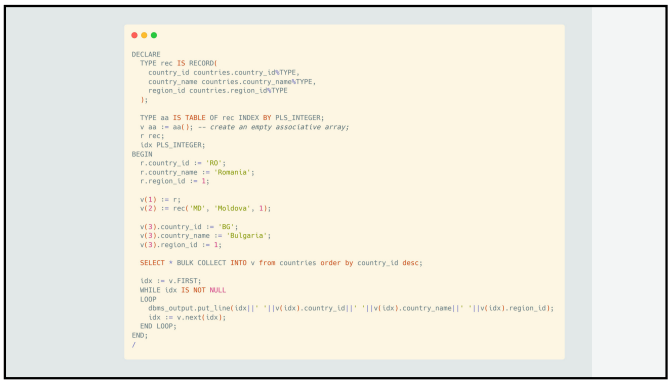
  v := aa(1 => r, 2 => rec('MD', 'Moldova', 1));

  v(5).country_id := 'BG';
  v(5).country_name := 'Bulgaria';
  v(5).region_id := 1;

  idx := v.FIRST;
  WHILE idx IS NOT NULL
  LOOP
    dbms_output.put_line(idx||' '||v(idx).country_id||' '
                                  ||v(idx).country_name||' '||v(idx).region_id);
    idxs(i) := idx;
    idx := v.next(idx);
    i := i + 1;
  END LOOP;

  FORALL j IN VALUES OF idxs INSERT INTO COUNTRIES VALUES v(j);

END;
/
```



```
DECLARE
  TYPE rec IS RECORD(
    country_id countries.country_id%TYPE,
    country_name countries.country_name%TYPE,
    region_id countries.region_id%TYPE
  );

  TYPE aa IS TABLE OF rec INDEX BY PLS_INTEGER;
  v aa := aa(); -- create an empty associative array;
  r rec;
  idx PLS_INTEGER;
BEGIN
  r.country_id := 'RO';
  r.country_name := 'Romania';
  r.region_id := 1;

  v(1) := r;
  v(2) := rec('MD', 'Moldova', 1);

  v(3).country_id := 'BG';
  v(3).country_name := 'Bulgaria';
  v(3).region_id := 1;

  SELECT * BULK COLLECT INTO v from countries order by country_id desc;

  idx := v.FIRST;
  WHILE idx IS NOT NULL
  LOOP
    dbms_output.put_line(idx||' '||v(idx).country_id||' '||v(idx).country_name||' '||v(idx).region_id);
    idx := v.next(idx);
  END LOOP;
END;
/
```

VARRARYS

A **varray** (**variable-size** array) is an array whose **number** of **elements** can vary from zero (empty) to the **declared** maximum size. An **uninitialized** varray variable is a **null collection**. A **varray** variable must be **initialized** by either making it **empty** or assigning a **non-NULL** value to it.

A **varray** is **appropriate** when the maximum number of needed elements is **known** and the elements are **accessed sequentially**. A **varray** might be **impractical** for storing **many** elements because all elements must be stored or retrieved at the same time.

VARRARYS - Index

Use the syntax "variable_name(index)" to **access** an **element** of a **varray** variable, where the **lower bound** of **index** is **1** and the **upper bound** is the **current number** of **elements**.

The **upper bound changes** as you add or delete **elements but** cannot exceed the maximum size.

```
DECLARE
  TYPE vec1 IS VARRAY(150) OF countries.country_id%type;
  v1 vec1 := vec1('RO', 'MD');
  TYPE vec2 IS VARRAY(150) OF countries.country_name%type;
  v2 vec2 := vec2('Romania', 'Moldova');
BEGIN
  FORALL i IN v1.FIRST .. v1.LAST
    INSERT INTO countries(country_id) VALUES(v1(i));

  FORALL i IN v2.FIRST .. v2.LAST
    UPDATE countries SET country_name=v2(i), region_id=1 WHERE country_id = v1(i);

  SELECT country_id, country_name
  BULK COLLECT INTO v1, v2
  FROM countries
  WHERE country_id NOT IN (SELECT DISTINCT country_id FROM locations WHERE country_id IS NOT
NULL);
  FOR i IN v1.FIRST .. v1.LAST
  LOOP
    dbms_output.put_line(v1(i)||' '||v2(i));
  END LOOP;

  FORALL i IN v1.FIRST .. v1.LAST
    DELETE countries WHERE country_id = v1(i);

END;
/
```



Nested Tables

Nested Tables

A **nested table** is a collection type that stores an **unspecified number** of rows in **no particular order**. When retrieving a **nested table** value from the **database** PL/SQL gives the rows **consecutive indexes** starting at **1**.

The syntax is "variable_name(**index**)".

A **nested table**'s **indexes** and **row order** might **not remain stable** when **storing** or **retrieving** the **nested table** to and from the **database**.

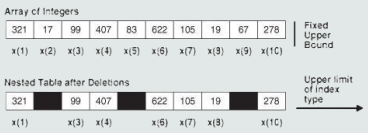
The amount of **memory** that a nested table variable occupies can **increase** or **decrease dynamically** as elements are added or deleted.

An **uninitialized** nested table variable is a **null collection**. A **nested table** is initialized by either making it **empty** or assigning a **non-NULL** value to it.

Nested Tables

Conceptually a **nested table** is like a one-dimensional **array** with an **arbitrary** number of elements, however, there are **differences**:

- The **size** of a nested table can **increase dynamically**: an **array** has a **declared** (**maximum**) number of elements, a nested table **does not**
- A **nested table** is **dense initially** but can become **sparse** as elements are **deleted** (at arbitrary indexes), whereas the **array** remains **dense**



Nested Tables

A **nested table** is **appropriate** when:

- The (maximum)number of elements is **not set**
- **Index** values are **not consecutive**
- To **delete** or **update some elements**, but **not all** elements **simultaneously**

```
DECLARE
  TYPE tbl1 IS TABLE OF countries.country_id%type;
  t1 tbl1 := tbl1('RO', 'MD');
  TYPE tbl2 IS TABLE OF countries.country_name%type;
  t2 tbl2 := tbl2('Romania', 'Moldova');
BEGIN
  FORALL i IN INDICES OF t1 INSERT INTO countries(country_id) VALUES(t1(i));

  FORALL i IN INDICES OF t2 UPDATE countries SET country_name=t2(i), region_id=1 WHERE country_id =
t1(i);

  SELECT country_id, country_name
  BULK COLLECT INTO t1, t2 FROM countries
  WHERE country_id NOT IN (SELECT DISTINCT country_id FROM locations WHERE country_id IS NOT NULL);

  FOR i IN t1.FIRST .. t1.LAST
  LOOP
    dbms_output.put_line(t1(i)||' '||t2(i));
  END LOOP;

  FORALL i IN INDICES OF t1 DELETE countries WHERE country_id = t1(i);

END;
/
```

Collection Constructors

A **collection constructor** (constructor) is a system-defined **function** that has the **same name** as a collection type and **returns** a collection of that type.

```
-- declare the type

TYPE vec IS VARRAY(10) OF INTEGER;

-- initialize an empty variable

var vec := vec();
```

Assigning Values to Collection Variables

- **Constructor**: Invoke a **constructor** to **create** a collection and **assign values** to the collection **elements**
- **Assignment**: To assign the value of **another** existing **collection** variable
- **Subprogram**: **Pass** the collection variable to a **subprogram** / Use the **OUT** or **IN OUT** parameter / **Assign** the value **inside** the **subprogram**

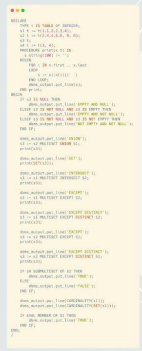
A collection can be assigned to a collection variable if they have the same data type. Having the same element type is not enough for an assignment.

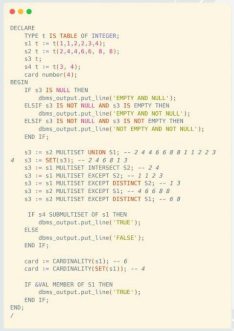
Assigning Values to Collection Variables

To make a **collection** variable **null** assign either a **NULL value** or a **NULL collection** of the same data type.

The **SQL MULTISET** operators (MULTISET **EXCEPT**, MULTISET **INTERSECT**, MULTISET **UNION**) **combine** two **nested tables** into a **single** nested table. The elements of the two **nested table**s must have **comparable** data types.

The result of the **SQL SET** function invocation takes a **nested table** argument and returns a nested table of the **same data type** whose elements are **distinct**. The function **eliminates duplicate** elements.





Collection Methods

A **collection method** is a PL/SQL **subprogram**, either a **function** that **returns information** about a collection or a **procedure** that **operates** on a collection.

Collection methods make collections easier to use and applications easier to maintain. The syntax for a method invocation is "collection_name.method".

Method	Type	Description
DELETE	Procedure	Deletes elements from collection.
TRIM	Procedure	Deletes elements from end of varray or nested table.
EXTEND	Procedure	Adds elements to end of varray or nested table.
EXISTS	Function	Returns TRUE if and only if specified element of varray or nested table exists.
FIRST	Function	Returns first index in collection.
LAST	Function	Returns last index in collection.
COUNT	Function	Returns number of elements in collection.
LIMIT	Function	Returns maximum number of elements that collection can have.
PRIOR	Function	Returns index that precedes specified index.
NEXT	Function	Returns index that succeeds specified index.

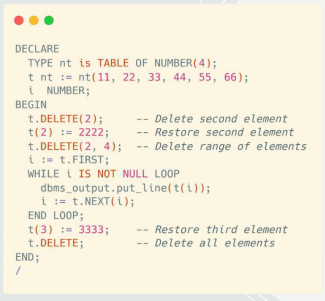
Collection Methods - **DELETE**

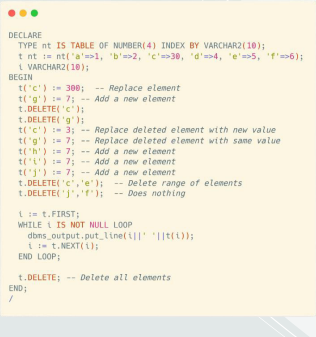
DELETE is a **procedure** that removes elements from a **collection**. "**DELETE**" **removes all** elements from a collection of **any type**. This operation **immediately** frees the memory allocated by the deleted elements.

For an **associative array** or **nested table** (but **not** a **varray**):

- **DELETE**(n) **removes the** element whose **index** is **n** if that element exists, otherwise it does nothing
- **DELETE**(m,n) **removes all** elements whose **indexes** are in the **range** m..n if both m and n exist and m <= n, otherwise it does nothing

For the above forms of **DELETE** PL/SQL keeps **placeholders** for the deleted elements. The deleted elements are included in the internal size of the collection, and they can be "restored" by assigning a valid value to it. (the cell, not the value of the element)





Collection Methods - **TRIM**

TRIM is a **procedure** that **deletes** elements from the **end** of a **varray** or **nested table**. This method has these forms:

- **TRIM** removes **one element** from the **end** of the collection if the collection has at least one element, otherwise it **raises** the predefined exception **SUBSCRIPT_BEYOND_COUNT**
- **TRIM**(n) removes **n elements** from the **end** of the collection if there are at least n elements at the end, otherwise it **raises** the predefined exception **SUBSCRIPT_BEYOND_COUNT**

TRIM operates on the **internal size** of a collection. It considers the element to exist: if **DELETE** removed an element but **kept** a **placeholder** for it, **TRIM** does a **clean wipe**. PL/SQL **does not** keep **placeholders** for **trimmed** elements. (Trimmed elements are not included in the internal size of the collection; a trimmed element cannot be restored by assigning a valid value to it)

```
DECLARE
  TYPE nt is TABLE OF NUMBER(4);
  t nt := nt(1, 2, 3, 4, 5, 6);
  i  NUMBER;
BEGIN
  t.TRIM; -- Trim last element
  t.DELETE(4); -- Delete fourth element
  t.TRIM(2); -- Trim last two element
  i := t.FIRST;
  WHILE i IS NOT NULL LOOP
    dbms_output.put_line(t(i));
    i := t.NEXT(i);
  END LOOP;
END;
/
```

Collection Methods - **EXTEND**

EXTEND is a **procedure** that **adds elements** to the **end** of a **varray** or **nested table**. The collection **can be empty**, but **not null** (Use a constructor to make a collection empty or add elements to a null collection). The **EXTEND** method has these forms:

- **EXTEND**: appends **one null** element to the end of the collection
- **EXTEND**(n): appends **n null** elements to the end of the collection
- **EXTEND**(n,i): appends **n copies** of the **i-th elemen**t to the collection. Is the **only** form that you can use for a collection whose elements have the **NOT NULL** constraint.

EXTEND operates on the **internal size** of a collection, it considers the element **to exist**, like **TRIM**, unlike **DELETE**.

```
DECLARE
  TYPE nt is TABLE OF NUMBER(4);
  t nt := nt(1, 2, 3, 4, 5, 6);
  i  NUMBER;
BEGIN
  t.EXTEND(2, 3); -- Append two copies of first element
  t.DELETE(8); -- Delete eighth element
  t.EXTEND; -- Append one null element
  i := t.FIRST;
  WHILE i IS NOT NULL LOOP
    dbms_output.put_line(i||' '||t(i));
    i := t.NEXT(i);
  END LOOP;
END;
/
```

Collection Methods - **EXISTS**

EXISTS is a **function** that tells you whether the specified element of a **varray** or **nested table** exists.

EXISTS(n): returns **TRUE** if the **n-th element** of the collection exists and **FALSE** otherwise.

If **n** is **out of range EXISTS** returns **FALSE** instead of raising the predefined exception **SUBSCRIPT_OUTSIDE_LIMIT**.

EXISTS(n) returns **FALSE** for a **deleted element** (even if **DELETE** kept a **placeholder** for it).

```
DECLARE
  TYPE rec IS RECORD(
      country_name countries.country_name%TYPE,
      region_name regions.region_name%TYPE
    );
  TYPE tbl IS TABLE OF rec;
  t tbl;
BEGIN

  SELECT country_name, region_name
  BULK COLLECT INTO t
  FROM countries natural join regions;

  dbms_output.put_line(t.FIRST); -- 1
  dbms_output.put_line(t.LAST); -- 25
  dbms_output.put_line(t.COUNT); -- 25

  t.DELETE(5, 15); -- Delete range

  FOR i IN t.FIRST .. t.LAST
  LOOP
    IF t.EXISTS(i) THEN -- test if exists
      dbms_output.put_line(t(i).country_name||' '||t(i).region_name);
    END IF;
  END LOOP;

  dbms_output.put_line(t.FIRST); -- 1
  dbms_output.put_line(t.LAST); -- 25
  dbms_output.put_line(t.COUNT); -- 14

END;
/
```

Collection Methods – **COUNT** and **LIMIT**

COUNT is a **function** that returns the **number of elements** in the **collection**. **COUNT ignores** deleted elements, even if **DELETE** kept **placeholders** for them.

For a **nested table COUNT** is smaller than **LAST** (the index of the **last** element) if elements from the **middle** of the nested table are **deleted**.

https://pt.memedroid.com/memes/detail/3965907/i-am-at-my-limit-too

LIMIT is a function that returns the **maximum number** of elements that the collection can have.

LIMIT returns **NULL** if the collection has no maximum number of elements. (Only a varray has a maximum size)

Collection Methods – **FIRST** and **LAST**

FIRST and **LAST** are functions. If the **collection** has at least one element **FIRST** return the **index** of the first element and **LAST** returns the **index** of the last element. They **ignore deleted** elements even if **DELETE** kept **placeholders**. **FIRST** and **LAST** return the same **index** if the collection has only one element. **FIRST** and **LAST** return **NULL** if the collection is **empty**.

- For an **associative array**, indexed by **PLS_INTEGER**: The **FIRST** element is the **smallest index** and the **LAST** element is the **largest index**
- For an **associative array**, indexed by **string**: The **FIRST** element is the **lowest key value** and the **LAST** element is the **highest key value**
- For a **varray** that is **not empty**: **FIRST** always returns **1** and **LAST** always equals **COUNT**
- For a **nested table**: **LAST** equals **COUNT** if elements from the **middle** of the nested table are **NOT deleted** and **LAST** is larger than **COUNT** if elements from its middle are **deleted**.

```
DECLARE
  TYPE rec IS RECORD(
      country_name countries.country_name%TYPE,
      region_name regions.region_name%TYPE
    );
  TYPE tbl IS TABLE OF rec;
  t tbl;
BEGIN

  SELECT country_name, region_name
  BULK COLLECT INTO t
  FROM countries natural join regions;

  dbms_output.put_line(t.FIRST); -- 1
  dbms_output.put_line(t.LAST); -- 25
  dbms_output.put_line(t.COUNT); -- 25

  t.DELETE(5, 15); -- Delete range

  FOR i IN t.FIRST .. t.LAST
  LOOP
    IF t.EXISTS(i) THEN -- test if exists
      dbms_output.put_line(t(i).country_name||' '||t(i).region_name);
    END IF;
  END LOOP;

  dbms_output.put_line(t.FIRST); -- 1
  dbms_output.put_line(t.LAST); -- 25
  dbms_output.put_line(t.COUNT); -- 14

END;
/
```

```
DECLARE
  TYPE tbl IS TABLE OF countries.country_name%TYPE;
  TYPE vec IS VARRAY(100) OF regions.region_name%TYPE;
  TYPE rec IS RECORD(
    country_names tbl,
    region_names vec
    );
  r rec;
BEGIN

  SELECT country_name
  BULK COLLECT INTO r.country_names
  FROM countries;

  SELECT region_name
  BULK COLLECT INTO r.region_names
  FROM regions;

  dbms_output.put_line('country_names.FIRST '||r.country_names.FIRST); -- 1
  dbms_output.put_line('country_names.LAST '||r.country_names.LAST); -- 25
  dbms_output.put_line('country_names.COUNT '||r.country_names.COUNT); -- 25
  dbms_output.put_line('country_names.LIMIT '||r.country_names.LIMIT); -- NULL

  dbms_output.put_line('region_names.FIRST '||r.region_names.FIRST); -- 1
  dbms_output.put_line('region_names.LAST '||r.region_names.LAST); -- 4
  dbms_output.put_line('region_names.COUNT '||r.region_names.COUNT); -- 4
  dbms_output.put_line('region_names.LIMIT '||r.region_names.LIMIT); -- 100

  FOR i IN r.country_names.FIRST .. r.country_names.LAST
  LOOP
    IF r.country_names.EXISTS(i) THEN -- test if exists
      dbms_output.put_line(r.country_names(i));
    END IF;
  END LOOP;
END;
/
```

Collection Methods – **PRIOR** and **NEXT**

PRIOR and **NEXT** are **functions** that let you **move backward** and **forward** in the collection. They **ignore** deleted elements, even if **DELETE** kept **placeholders** for them and are useful for **traversing sparse** collections. Given an **index** (**must exist**):

- **PRIOR**: If it **exists**, then **PRIOR** returns the index of the **preceding existing** element else **PRIOR** returns **NULL**
- **NEXT**: If it **exists**, then **NEXT** returns the index of the **succeeding existing** element else **NEXT** returns **NULL**

For an **associative array indexed** by **string** PRIOR and **NEXT** indexes are determined by key values (always sorted order).

If the collection c is a varray and the index exceeds c.LIMIT then c.PRIOR(index) returns c.LAST and c.NEXT(index) returns NULL.

c.PRIOR(c.FIRST) returns NULL for any collection c

c.NEXT(c.LAST) returns NULL for any collection c

```
DECLARE
    TYPE column IS TABLE OF INTEGER INDEX BY
PLS_INTEGER;
    TYPE matrix IS TABLE OF column INDEX BY PLS_INTEGER;
    mat row;
    tmat row;
    i STRING(100);
    j PLS_INTEGER;
    j PLS_INTEGER;
BEGIN
    mat := row( 1 => column(1=>1, 2=> 2, 3=> 3, 4=> 4),
                2 => column(1=>5, 2=> 6, 3=> 7, 4=> 8),
                3 => column(1=>9, 2=>10, 3=>11, 4=>12));

    dbms_output.put_line('Original Matrix');
    i := mat.LAST;
    WHILE i IS NOT NULL
    LOOP
        j := mat(i).LAST;
        WHILE j IS NOT NULL
        LOOP
            tmat(j)(i) := mat(i)(j);
            j := mat(i).PRIOR(j);
        END LOOP;
        i := mat.PRIOR(i);
    END LOOP;

    dbms_output.put_line('Transpose Matrix');
    i := tmat.FIRST;
    WHILE i IS NOT NULL
    LOOP
        j := '';
        j := tmat(i).FIRST;
        WHILE j IS NOT NULL
        LOOP
            j := j || tmat(i)(j) || ' ';
            j := tmat(i).NEXT(j);
        END LOOP;
        dbms_output.put_line(i);
        i := tmat.NEXT(i);
    END LOOP;

END;
/
```

```
CREATE OR REPLACE TYPE employees_vec AS VARRAY(20) OF varchar2(20);
/

CREATE TABLE EmployeesDeparments(
  numedept varchar2(30),
  angajati Employees_vec
);

DECLARE
  emps employees_vec;
  idd departments.department_id%TYPE := 110;
  dname departments.department_name%TYPE;
BEGIN
  SELECT department_name INTO dname FROM departments WHERE department_id =
idd;
  SELECT first_name||' '||last_name
  BULK COLLECT INTO emps
  FROM employees
  WHERE department_id = idd;

  INSERT INTO EmployeesDeparments values(dname, emps);
END;
/

SELECT * FROM EmployeesDeparments;

DROP TABLE EmployeesDeparments;
DROP TYPE employees_vec;
```

```
CREATE OR REPLACE TYPE jobs_nt AS TABLE OF varchar2(30);
/

CREATE TABLE EmployeesJobs(
  ename varchar2(40),
  ejobs jobs_nt
)
NESTED TABLE ejobs STORE AS ejobs_store;

set serveroutput on;
DECLARE
  ejobs jobs_nt;
  ide employees.employee_id%TYPE := 101;
  ename varchar2(40);
BEGIN
  SELECT first_name||' '||last_name INTO ename
  FROM employees WHERE employee_id = ide;

  WITH
    q1 as (SELECT e.employee_id, j.job_title
           FROM employees e
                INNER JOIN job_history jh ON jh.employee_id=e.employee_id
                INNER JOIN jobs j ON j.job_id = jh.job_id
          UNION
          SELECT e.employee_id, j.job_title
          FROM employees e
               INNER JOIN jobs j ON j.job_id = e.job_id)
  SELECT job_title BULK COLLECT INTO ejobs
  FROM q1
  WHERE q1.employee_id = ide;

  INSERT INTO EmployeesJobs values(ename, ejobs);

END;
/

SELECT * FROM EmployeesJobs;

DROP TABLE EmployeesJobs;

DROP TYPE jobs_nt;
```